Signed and Unsigned Numbers

Unsigned binary integers

- Given an n-bit number

$$\boxed{x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0}$$

- Range: 0 to $+2^n - 1$
- Example
  - $0000\ 0000 \cdots 0000\ 1011_2$

    $= 0 + \cdots + 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0$

    $= 0 + \cdots + 8 + 0 + 2 + 1 = 11_{10}$
- Using 32 bits
  - 0 to +4,294,967,295
- Using 64 bits
  - 0 ~ +18,446,774,073,709,551,615

2s-Complement Signed Integers

- Given an n-bit number

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: -2^{n-1} to $+2^{n-1} - 1$
- Example
  - $1111\ 1111 \cdots 1111\ 1100_2$
    $= -1\times2^{31} + 1\times2^{30} + \cdots + 1\times2^2 + 0\times2^1 + 0\times2^0$
    $= -2{,}147{,}483{,}648 + 2{,}147{,}483{,}644 = -4_{10}$
- Using 32 bits
  - -2,147,483,648 to +2,147,483,647
- Using 64 bits
  - -9,223,372,036,854,775,808 ~ +9,223,372,036,854,775,807

2s-Complement Signed Integers

- MSB bit is a sign bit
  - 1 for negative numbers
  - 0 for non-negative numbers
- $-(-2^{n-1})$ can't be represented
- Non-negative numbers have the same unsigned and 2s-complement representation
- Some specific numbers
  - 0: 0000 0000 ··· 0000
  - -1: 1111 1111 ··· 1111
  - Most-negative: 1000 0000 ··· 0000
  - Most-positive: 0111 1111 ··· 1111

Signed Negation

- Complement and add 1
  - Complement means 1 → 0, 0 → 1
- Example: negate +2
  - +2 = 0000 0000 ⋯ 0010_2
  - -2 = 1111 1111 ⋯ 1101_2 + 1

    = 1111 1111 ⋯ 1110_2

$$\boxed{\begin{aligned} x + \bar{x} &= 1111\ldots111_2 = -1 \\ \bar{x} + 1 &= -x \end{aligned}}$$

Sign Extension

- Representing a number using more bits
  - Preserve the numeric value
- Replicate the sign bit to the left
  - c.f. unsigned values: extend with 0s
- Examples: 8-bit to 16-bit
  - +2: 0000 0010 => 0000 0000 0000 0010
  - -2: 1111 1110 => 1111 1111 1111 1110
- In RISC-V instruction set
  - `lb`: sign-extend loaded byte
  - `lbu`: zero-extend loaded byte

Arithmetic Operations

Instruction	Type	Example	Meaning
Add	R	`add rd, rs1, rs2`	`R[rd] = R[rs1] + R[rs2]`
Subtract	R	`sub rd, rs1, rs2`	`R[rd] = R[rs1] - R[rs2]`
Add immediate	I	`addi rd, rs1, imm12`	`R[rd] = R[rs1] + SignExt(imm12)`
Set less than	R	`slt rd, rs1, rs2`	`R[rd] = (R[rs1] < R[rs2])? 1 : 0`
Set less than immediate	I	`slti rd, rs1, imm12`	`R[rd] = (R[rs1] < SignExt(imm12))? 1 : 0`
Load upper immediate	U	`lui rd, imm20`	`R[rd] = SignExt(imm20 << 12)`
Add upper immediate to PC	U	`auipc rd, imm20`	`R[rd] = PC + SignExt(imm20 << 12)`

Data Transfer Operations

Instruction	Type	Example	Meaning
Load doubleword	I	`ld rd, imm12(rs1)`	`R[rd] = Mem`$_8$`[R[rs1] + SignExt(imm12)]`
Load word	I	`lw rd, imm12(rs1)`	`R[rd] = SignExt(Mem`$_4$`[R[rs1] + SignExt(imm12)])`
Load halfword	I	`lh rd, imm12(rs1)`	`R[rd] = SignExt(Mem`$_2$`[R[rs1] + SignExt(imm12)])`
Load byte	I	`lb rd, imm12(rs1)`	`R[rd] = SignExt(Mem`$_1$`[R[rs1] + SignExt(imm12)])`
Store doubleword	S	`sd rs2, imm12(rs1)`	`Mem`$_8$`[R[rs1] + SignExt(imm12)] = R[rs2]`
Store word	S	`sw rs2, imm12(rs1)`	`Mem`$_4$`[R[rs1] + SignExt(imm12)] = R[rs2](31:0)`
Store halfword	S	`sh rs2, imm12(rs1)`	`Mem`$_2$`[R[rs1] + SignExt(imm12)] = R[rs2](15:0)`
Store byte	S	`sb rs2, imm12(rs1)`	`Mem`$_1$`[R[rs1] + SignExt(imm12)] = R[rs2](7:0)`

Encoding Instructions

```
add   x9, x20, x21
```

$$0000\ 0001\ 0101\ 1010\ 0000\ 0100\ 1011\ 0011_2 = 0x015A04B3$$

Representing Instructions

- Instructions are encoded in binary
  - Called machine code
- RISC-V instructions
  - Encoded as 32-bit instruction words
  - Small number of formats encoding operation code (opcode), register numbers, …
  - Regularity!

RISC-V R-format Instructions

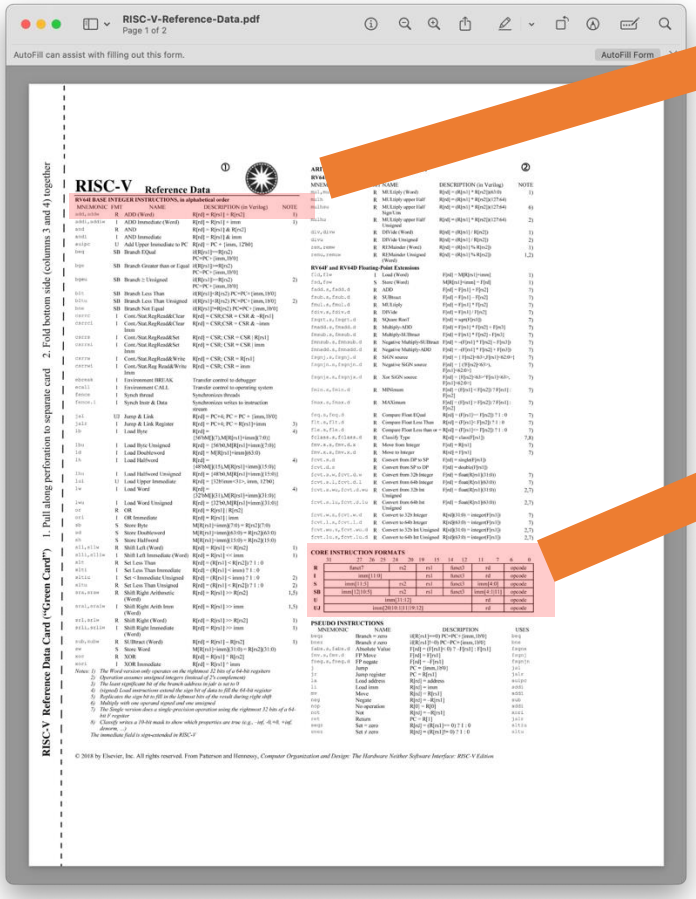
funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

- Instruction fields
  - `opcode`: operation code
  - `rd`: destination register number
  - `funct3`: 3-bit function code (additional opcode)
  - `rs1`: first source register number
  - `rs2`: second source register number
  - `funct7`: 7-bit function code (additional opcode)

Example: add instruction (1/2)

```
add   x9, x20, x21
```

1. Refer to the ISA manual (e.g., RISC-V green sheet)



2. Look up the table for add instruction

RV64I BASE INTEGER INSTRUCTIONS, in alphabetical order

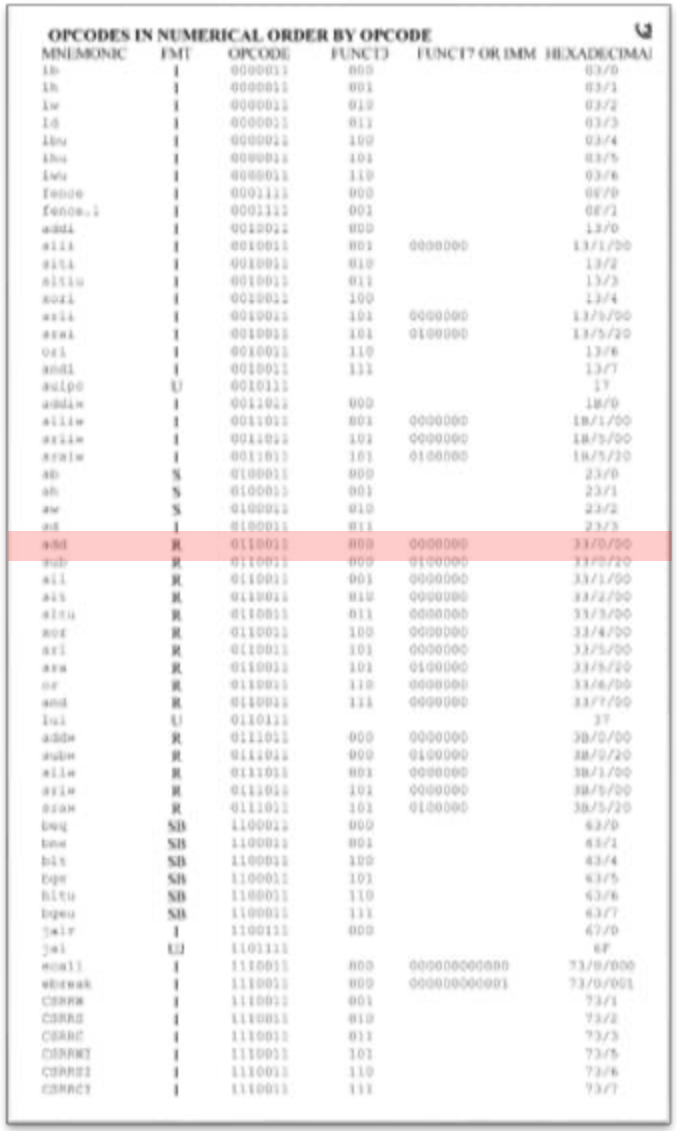
MNEMONIC	FMT	NAME	DESCRIPTION (in Verilog)	NOTE
add, addw	R	ADD (Word)	R[rd] = R[rs1] + R[rs2]	1)

3. Encode the instruction by following the format

CORE INSTRUCTION FORMATS

	31 27	26 25	24 20	19 15	14 12	11 7	6 0
R	funct7		rs2	rs1	funct3	rd	opcode
I	imm[11:0]			rs1	funct3	rd	opcode
S	imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode
SB	imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]					rd	opcode
UJ	imm[20|10:1|11|19:12]					rd	opcode

Example: add instruction (1/2)



OPCODES IN NUMERICAL ORDER BY OPCODE

MNEMONIC	FMT	OPCODE	FUNCT3	FUNCT7 OR IMM	HEXADECIMAL
lb	I	0000011	000		03/0
lh	I	0000011	001		03/1
lw	I	0000011	010		03/2
ld	I	0000011	011		03/3
lbu	I	0000011	100		03/4
lhu	I	0000011	101		03/5
lwu	I	0000011	110		03/6
fence	I	0001111	000		0F/0
fence.i	I	0001111	001		0F/1
addi	I	0010011	000		13/0
slli	I	0010011	001	0000000	13/1/00
slti	I	0010011	010		13/2
sltiu	I	0010011	011		13/3
xori	I	0010011	100		13/4
srli	I	0010011	101	0000000	13/5/00
srai	I	0010011	101	0100000	13/5/20
ori	I	0010011	110		13/6
andi	I	0010011	111		13/7
auipc	U	0010111			17
addiw	I	0011011	000		1B/0
slliw	I	0011011	001	0000000	1B/1/00
sriw	I	0011011	101	0000000	1B/5/00
sraiw	I	0011011	101	0100000	1B/5/20
sb	S	0100011	000		23/0
sh	S	0100011	001		23/1
sw	S	0100011	010		23/2
sd	I	0100011	011		23/3
add	R	0110011	000	0000000	33/0/00
sub	R	0110011	000	0100000	33/0/20
sll	R	0110011	001	0000000	33/1/00
slt	R	0110011	010	0000000	33/2/00
sltu	R	0110011	011	0000000	33/3/00
xor	R	0110011	100	0000000	33/4/00
srl	R	0110011	101	0000000	33/5/00
sra	R	0110011	101	0100000	33/5/20
or	R	0110011	110	0000000	33/6/00
and	R	0110011	111	0000000	33/7/00
lui	U	0110111			37
addw	R	0111011	000	0000000	3B/0/00
subw	R	0111011	000	0100000	3B/0/20
sllw	R	0111011	001	0000000	3B/1/00
srlw	R	0111011	101	0000000	3B/5/00
sraw	R	0111011	101	0100000	3B/5/20
beq	SB	1100011	000		63/0
bne	SB	1100011	001		63/1
blt	SB	1100011	100		63/4
bge	SB	1100011	101		63/5
bltu	SB	1100011	110		63/6
bgeu	SB	1100011	111		63/7
jalr	I	1100111	000		67/0
jal	UJ	1101111			6F
ecall	I	1110011	000	000000000000	73/0/000
ebreak	I	1110011	000	000000000001	73/0/001
CSRRW	I	1110011	001		73/1
CSRRS	I	1110011	010		73/2
CSRRC	I	1110011	011		73/3
CSRRWI	I	1110011	101		73/5
CSRRSI	I	1110011	110		73/6
CSRRCI	I	1110011	111		73/7

```
add    x9, x20, x21
```

MNEMONIC	FMT	OPCODE	FUNCT3	FUNCT7 OR IMM	HEXADECIMAL
add	R	0110011	000	0000000	33/0/00

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
0	x21	x20	0	x9	add
0	21	20	0	9	51
0000000	10101	10100	000	01001	0110011

$0000\ 00001\ 0101\ 1010\ 0000\ 0100\ 1011\ 0011_2 = 0x015A04B3$

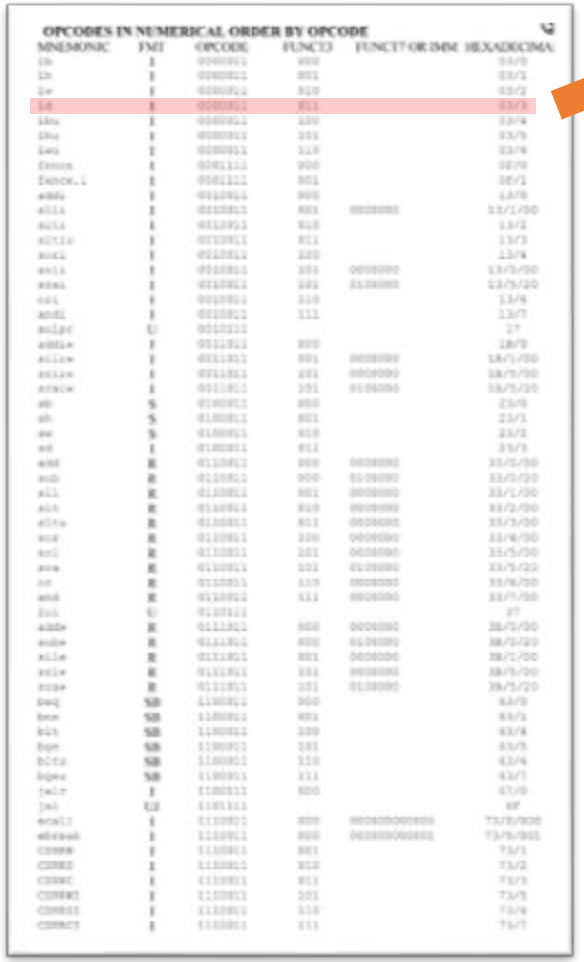
RISC-V I-format Instructions

immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

- Immediate arithmetic and load instructions
  - `rs1`: source or base address register number
  - `immediate`: constant operand, or offset added to base address
    - 2s-complement, sign extended
- **Design Principle 4: Good design demands good compromises**
  - Different formats complicate decoding, but allow 32-bit instructions uniformly
  - Keep formats as similar as possible

Example: ld instruction

```
ld    x9,   64(x22)
```



MNEMONIC	FMT	OPCODE	FUNCT3	FUNCT7 OR IMM	HEXADECIMAL
lb	I	0000011	000		03/0
lh	I	0000011	001		03/1
lw	I	0000011	010		03/2
ld	I	0000011	011		03/3

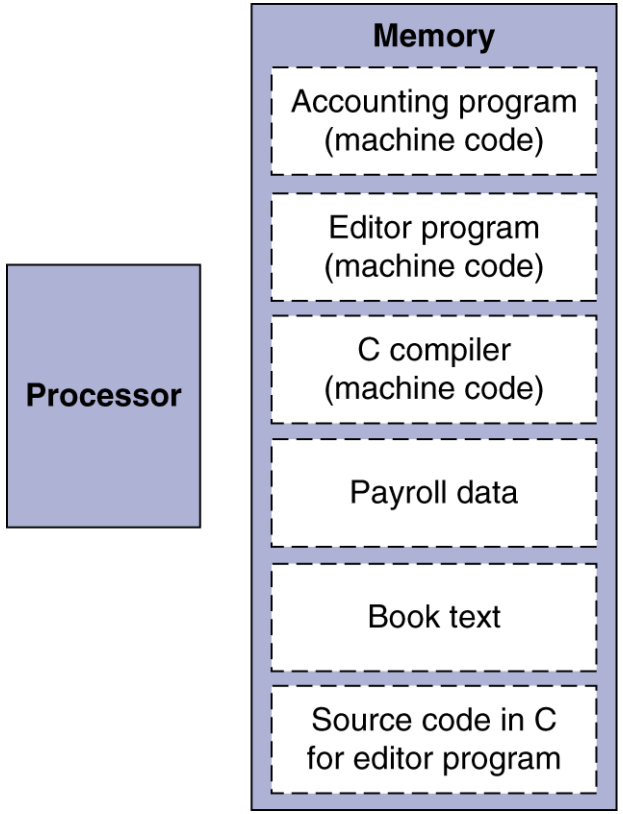
immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits
64	22	3	9	3
0000 0100 0000	10110	011	01001	0000011

RISC-V S-format Instructions

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

- Different immediate format for store instructions
  - `rs1`: base address register number
  - `rs2`: source operand register number
  - `immediate`: offset added to base address
    - Split so that `rs1` and `rs2` fields always in the same place

Stored Program Computers



- Instructions represented in binary, just like data
- Instructions and data stored in memory
- Programs can operate on programs
  - e.g., compilers, linkers, …
- Binary compatibility allows compiled programs to work on different computers
  - Standardized ISAs

Logical Operations

Logical Operations

- Instructions for bitwise manipulation

Operation	C	Java	RISC-V
Shift left	<<	<<	`slli`
Shift right	>>	>>>	`srli`
Bitwise AND	&	&	`and, andi`
Bitwise OR	|	|	`or, ori`
Bitwise XOR	^	^	`xor, xori`
Bitwise NOT	~	~	

- Useful for extracting and inserting groups of bits in a word

`Shift` Operations

`I-format`

`immediate`	`rs1`	`funct3`	`rd`	`opcode`
12 bits	5 bits	3 bits	5 bits	7 bits

- `immed`: how many positions to shift
- Shift left logical
  - Shift left and fill with 0 bits
  - `slli` by i bits multiples by 2^i
- Shift right logical
  - Shift right and fill with 0 bits
  - `srli` by i bits divides by 2^i (unsigned only)

AND Operations

- Useful to mask bits in a word
  - Select some bits, clear others to 0

```
and   x9, x10, x11
```

x10	0000 0000 0000 0000 0000 1100 0000 0000
x11	0000 0000 0000 0000 0011 1100 0000 0000
x9	0000 0000 0000 0000 0000 1100 0000 0000

OR Operations

- Useful to include bits in a word
  - Set some bits to 1, leave others unchanged

```
or    x9, x10, x11
```

x10	0000 0000 0000 0000 0000 1100 0000 0000
x11	0000 0000 0000 0000 0011 1100 0000 0000
x9	0000 0000 0000 0000 0011 1100 0000 0000

XOR Operations

- If both bits are same, then the result is 0 Otherwise, the result is 1

```
xor   x9, x10, x12
```

x10	0000 0000 0000 0000 0001 0100 0000 0000
x12	0000 0000 0000 0000 0010 1100 0000 0000
x9	0000 0000 0000 0000 0011 1000 0000 0000

NOT Operations

- Useful to invert bits in a word
  - Change 0 to 1, and 1 to 0
- NOT a = a XOR 1's

```
XOR    x9, x10, x12
```

x10	0000 0000 0000 0000 0001 0100 0000 0000
x12	1111 1111 1111 1111 1111 1111 1111 1111
x9	1111 1111 1111 1111 1110 1011 1111 1111

Instructions for Making Decisions

Conditional operations

- Branch to a labeled instruction if a condition is true
  - Otherwise, continue sequentially

```
beq   rs, rt, L1
```

- `if (rs == rt)` branch to instruction labeled `L1`

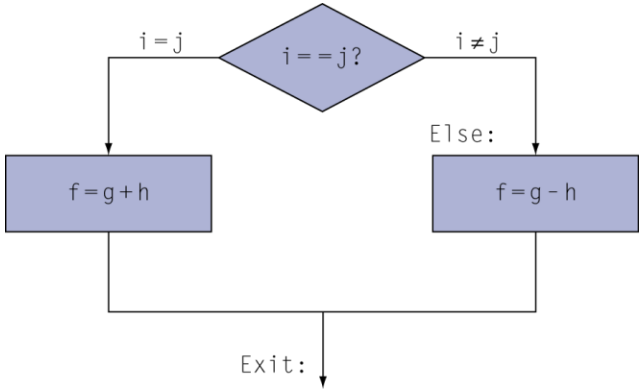
```
bne   rs, rt, L1
```

- `if (rs != rt)` branch to instruction labeled `L1`

Compiling `if` Statements

- C code:

```
if (i == j)
    f = g + h;
else
    f = g - h;
```



  - `f, g, h, i,` and `j` in `x19, x20, x21, x22,` and `x23`

- Compiled RISC-V code:

Assembler calculates addresses

Compiling `while` Statements

- C code:

```
while (save[i] == k)
      i += 1;
```

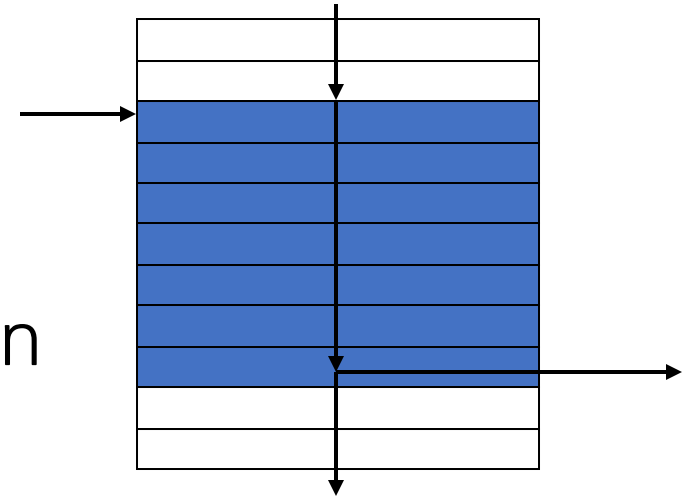
  - `i` in `x22`, `k` in `x24`, address of `save` in `x25`
  - Suppose the data type of the `save` array is doubleword (8B)

- Compiled RISC-V code:

```
Loop:     slli x10, x22, 3
          add  x10, x10, x25
          ld   x9,  0(x10)
          bne  x9,  x24, Exit
          addi x22, x22, 1
          beq  x0,  x0,  Loop
Exit:     ...
```

Basic blocks

- A basic block is a sequence of instructions with
  - No embedded branches (except at end)
  - No branch targets (except at beginning)
- A compiler identifies basic blocks for optimization
- An advanced processor can accelerate execution of basic blocks



More conditional operations

```
blt rs1, rs2, L1
```

- if (`rs1` < `rs2`) branch to instruction labeled `L1`

```
bge rs1, rs2, L1
```

- if (`rs1` >= `rs2`) branch to instruction labeled `L1`

- C code:

```
if (a > b)
      a += 1;
```

`a` in `x22`, `b` in `x23`

RISC-V code:

```
      bge  x23, x22, Exit
      addi x22, x22, 1
Exit:
```

Signed vs. Unsigned

- Signed comparison: `blt`, `bge`
- Unsigned comparison: `bltu`, `bgeu`

```
x22 = 1111 1111 1111 1111 1111 1111 1111 1111
x23 = 0000 0000 0000 0000 0000 0000 0000 0001
```

```
blt  x22, x23, Exit
```

= Go to Exit if -1 < 1

```
bltu x22, x23, Exit
```

= Go to Exit if 2^{64}-1 < 1

Branch Addressing

- Branch instructions specify
  - Opcode, two registers, target address
- Most branch targets are near branch
  - Forward or backward
- **SB** format:

imm[12:10-5]	rs2	rs1	funct3	imm[4-1:11]	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

`imm[12:1] << 1`: offset added to PC (two's complement, sign extended)

- PC-relative addressing
  - Target address = PC + immediate x 2

Jump Addressing

- Jump and link (`jal`) targets uses 20-bit immediate for larger range
- **`UJ`** format:

`imm[20:10-1:11:19-12]`	`rd`	`opcode`
20 bits	5 bits	7 bits

- For long jumps, e.g., to 32-bit absolute address
  - `lui`: load address [31:12] to temp register
  - `jalr`: add address [11:0] and jump to target

Branching Far Away

- What if the branch destination is further away than can be captured in 12 bits?
- The assembler comes to the rescue
  - It inserts an unconditional jump to the branch target and inverts the condition

```
beq  x10, x10, L1
```

⇩

```
     bne  x10, x0, L2
     jal  x0, L1
L2:
```

32-bit Constants

- Most constants are small
  - 12-bit immediate is sufficient

U-format

immediate[31:12]	rd	opcode
20 bits	5 bits	7 bits

- For the occasional 32-bit constant
  - Copies 20-bit constant to bits [31:12] of `rd`
  - Clear bits [11:0] of `rd` to 0

```
lui   rd,   constant
```

- Compiled RISC-V code:

```
lui   x19, 976          // 0x003D0
addi  x19, x19, 128     // 0x500
```

0000 0000 0011 1101 0000	0000 0000 0000
0000 0000 0011 1101 0000	0101 0000 0000

Control Transfer Instructions

Instruction	Type	Example	Meaning
Branch equal	SB	`beq rs1, rs2, imm12`	`if (R[rs1] == R[rs2])` `pc = pc + SignExt(imm12 << 1)`
Branch not equal	SB	`bne rs1, rs2, imm12`	`if (R[rs1] != R[rs2])` `pc = pc + SignExt(imm12 << 1)`
Branch greater than or equal	SB	`bge rs1, rs2, imm12`	`if (R[rs1] >= R[rs2])` `pc = pc + SignExt(imm12 << 1)`
Branch less than	SB	`blt rs1, rs2, imm12`	`if (R[rs1] < R[rs2])` `pc = pc + SignExt(imm12 << 1)`
Jump and link	UJ	`jal rd, imm20`	`R[rd] = pc + 4` `PC = PC + SignExt(imm20 << 1)`
Jump and link register	I	`jalr rd, imm12(rs1)`	`R[rd] = pc + 4` `PC = (R[rs1] + SignExt(imm12)) & (~1)`

Target Addressing Example

- Loop code from earlier example
  - Assume Loop at location 80000

```
Loop: slli x10, x22, 3
      add  x10, x10, x25
      ld   x9,  0(x10)
      bne  x9,  x24, Exit
      addi x22, x22, 1
      beq  x0,  x0,  Loop
Exit: …
```

80000	3		22	1	10	19
80004	0	25	10	0	10	51
80008	0		10	3	9	3
80012	0	24	9	1	12	99
80016	1		22	0	22	19
80020	127	0	0	0	13	99
80024						